

Feedback

Codebase Review & Technical Assessment

Preliminary Notes

Note on Local Environment Setup: I was unable to initially start the app in a working manner. This is because the Frontend was directly connected to a Supabase database I don't have access to.

To avoid making a new Supabase project, I changed this to a local database (and added a backend, as is best practice) for the time being. That way you can also see the most accurate version of my changes if I choose to edit anything on the database side.

Aside from the changes above, I wanted to note that I separated my thoughts into 3 categories: * **Dumpster Fire:** Things that will cause immediate, catastrophic failures to the app's safety and integrity. * **Bad Practice:** Things that are objectively wrong, but are not major concerns. * **Nit Picks:** Things I as a developer have seen work better from both a user and developer experience side of things.

Within each section, corrections are grouped by type: **Technical** refers to code, architecture, and implementation issues, while **Content** refers to UX, messaging, and information design issues.

Table of Contents

- [Preliminary Notes](#)
 -  [Dumpster Fire](#)
 - [Technical](#)
 - [Content](#)
 -  [Bad Practice](#)
 - [Technical](#)
 - [Content](#)
 -  [Nit Picks](#)
 - [Technical](#)
 - [Content](#)
-

Dumpster Fire

Critical issues affecting safety and integrity.

Technical

1. Direct Frontend-to-Database Connection TECHNICAL

Never, ever, ever (no not even then) connect your frontend directly to your database.

Reasoning: Your frontend should not only be treated as a simple app, but a gateway to the user's computer. This is because a user can change an app locally to reflect whatever app they want, causing it to be able to make whatever requests to your database (or they can just make the raw requests themselves from their computer, bypassing your pretty interface altogether).

This lets a user do everything to your database that you as a developer are able to do, which poses huge security and infrastructure issues as a user can just as easily request another user's password as drop a core table. You might finally argue that a database can be row-level protected, but by the time you're protecting from things like spam and security, you might as well have a dedicated backend.

Content

2. Missing the "So-What?" CONTENT

This app needs a punchy "so-what?" What is the killer page of this app?

Why do users log in? On social media, people spend most of their time on the "feed". On text and email apps, it's the text and email interface. More relevantly, on nutrition apps, it's the "log food, see progress" pages.

The lattermost is likely most relevant to this app where the initial page is logging caffeine, but the other pages get lost as to what the "progress" actually is. Is it to reduce further caffeine consumption? Is it to share caffeine consumption stats with fellow caffeine lovers?

I understand that the point of the app right now is to see how caffeine affects sleep, but without some measure of progress (or even regress), a user has no reason to continue logging into your app to see their stats. For example, my screen time page tells me "down/up X% from last week". That stat keeps me coming back so I can lower my screen time and is the killer feature of the screen time "app" Apple offers.

Bad Practice

Deviations from objective standards and best practices.

Technical

1. Overuse of `localStorage` TECHNICAL

Your app uses `localStorage` everywhere for maintaining and updating values locally. You should usually never do this unless you have a very good reason.

Reasoning: It's best practice to make each page retrieve information from the backend whenever the user navigates anywhere instead of pulling it from a local storage. This is because the backend could update without the frontend knowing, leading to a mismatch of states, like if the Oura Ring were to send an update while the user is on the app.

Also, local storage makes things “sticky”. In order to change anything the app needs to update both the local storage information and the database (like if the user adds caffeine consumed), which increases the vector of possible failure points and could lead to issues about which source to prioritize if one gets updated but not the other.

I do know of instances where `localStorage` is useful, like if the user is on a particular ‘project’ view of the same page, but often you want to follow the logic of “if it can be done in the backend, it should”.

2. Responsiveness TECHNICAL

Your app should be responsive to every screen size. * *Example of this being broken:* The “+ Log Caffeine” button popup in the /caffeine page is cut off on smaller phones, whereas it should be scrollable and made to fit with any phone.

3. Routing Architecture TECHNICAL

Your app uses conditional as opposed to URL routing. What this means (conditional routing) is there is an internal variable telling the app what page the user is on. URL routing on the other hand uses a specialized URL, like `caffeine.com/caffeine` or `caffeine.com/insights` for your different pages.

While the former works on an app, it makes it much harder to debug. Let's say a navigation button is broken and I just want to fix the next page. It's much easier for me to just change the URL to the page I want than to change some internal variable somewhere about what page I am on. On an app, the difference is negligible, but the main benefit is to the developer.

4. Authentication TECHNICAL

Your login process needs to work. This one is pretty self-explanatory. I added authentication logic to keep the user signed in.

5. Input Validation TECHNICAL

Your sign-up page should validate inputs. It needs to verify that I am inputting both a valid/new email and a password that meets some complexity guidelines (length, special characters, capitalization, etc.). Right now both allow any string.

Content

6. Excessive Clicks CONTENT

Users should click on the least amount of things possible to achieve a goal. * *Example of this being broken:* In the “+ Log Caffeine” button popup in the /caffeine page, the app should auto-select “morning”, “afternoon”, or “evening” based on the time of day it currently is, as a user is likely to log caffeine soon after intake.

7. Accessibility CONTENT

Be informationally inclusive by adding info tooltips. Most people know what sleep score means. But some don't. There is something called the Curb Cut Effect, which states when you design your product for disabled and less capable clientele, you make the experience better for everyone else. In your app, you should have a small tooltip info icon next to your sleep score which reveals what this score actually means. Those out of the know can easily figure out what this score means, and those in the know can read something that puts into context and heightens the importance of a sleep score.

Nit Picks

Technical

- **User Documentation (App-Wide) Technical:** You should have a basic README.md file describing the structure and setup of your app. Without this, developers can go down a rabbit whole trying to figure out what environmental variables (these are your hidden passwords), start commands for spinning up the app, and hardcoded things to include. These aren't available to developers by default because you should always keep your passwords private and there is no “start-up” page without a README.md.
- **AI Documentation (App-Wide) Technical:** If you are an AI coder, which it seems like you are based off your use of Bolt, you should have logic and rules for AI under every major folder. This is best practice because it allows the AI to use natural language to quickly understand what a portion of your code does and how not to break anything it is editing, or something tangential to what it is editing. I have a tool I coded up for auto-generating these rules that instruct your AI tool to refer to and update these rules: [AI Rules Generator](#)

Content

- **Oura Ring Integration Display CONTENT:** Remove this Oura Ring connected card. It takes up way too much space and the user knows it already if they've seen it once. Maybe put a small badge somewhere to confirm to the user the Oura Ring is connected with a tooltip menu for checking. This card should only be shown if the Oura Ring is **NOT** connected to prompt the user to get their data.
- **Data Visualization (Sleep) CONTENT:** Make the cards in the “Recent Sleep” section collapsible, or replace them with a bar chart a user can click into (like the Apple screen time functionality). This lets a user easily refer back to previous days, infer patterns, and see all their sleep data in one place.

- **Reasoning (Sleep) CONTENT:** Provide caffeine context for why a user's sleep might have been bad within this same card. If the point of the app is to show user's the effect of caffeine on their sleep, they should be able to dive into that.
- **Data Visualization (Caffeine vs. Sleep) CONTENT:** This chart is unclear/unhelpful in a lot of ways.
 - If this is relating caffeine consumption in a day to sleep quality, how is it distinguishing between coffee, tea, and other, if a user can have all three?
 - If the point of it is to convey the effects of caffeine on sleep quality, wouldn't a user downloading this app know that already and wouldn't the results match the statistical average decline for this kind of thing?
 - If not to get a user to drink less caffeine, what is the actionable response to this graph?
 - Finally, the graph is formatted poorly for mobile and might be better served with less horizontal padding and a feature to allow the user to click any dot to see the source data point.